

IoT Course — Hands-On #4

White Cube Device Emulators — Assignment #2

Student: Yakira Siboni

Broker: broker.hivemq.com | Port: 1883

1. Overview

This report documents the implementation and testing of IoT device emulators using the PyQt5 GUI framework and MQTT protocol. Each emulator simulates a physical IoT device — a DHT temperature/humidity sensor, a Relay switch, and a Push Button — communicating through the HiveMQ public MQTT broker.

All emulators are written in Python using the paho-mqtt library and connect to broker.hivemq.com on port 1883. The mqtt_init.py configuration file manages broker settings centrally across all scripts.

File	Device Type	Role	Topic
DHT.py	Temperature & Humidity sensor emulator	Publishes every 5 sec	pr/home/5976397/sts
RELAY.py	Relay switch emulator	Subscribes & shows ON/OFF status	pr/home/<random>/sts
BUTTON.py	Push button emulator	Publishes on button press	pr/home/button_123_YY/sts

2. System Setup

2.1 Configuration — mqtt_init.py

The mqtt_init.py file centralizes all broker connection parameters. Setting nb=1 selects the public HiveMQ broker, which was used throughout all tests.

- Broker: broker.hivemq.com
- Port: 1883 (standard MQTT)
- Authentication: none required for HiveMQ public broker
- Username/password: empty strings for HiveMQ

2.2 How the Emulators Work

Each emulator (DHT.py, RELAY.py, BUTTON.py) opens a PyQt5 GUI window with an "Enable/Connect" button. Clicking it establishes the MQTT connection to the broker. From that point:

- DHT.py — fires a QTimer every 5 seconds, generates random temperature (22-23°C) and humidity (74-99%) values, and publishes them to its topic
- RELAY.py — subscribes to its topic and toggles the Status button color between gray (OFF) and red (ON) whenever a message arrives
- BUTTON.py — publishes a message to its topic every time the "PUSH BUTTON" button is clicked

3. Test Results

Test 1 — DHT Sensor Data Collection

Goal: Subscribe to the DHT emulator topic and collect 20 temperature/humidity samples

Emulator used: DHT.py

Topic: pr/home/5976397/sts

Broker: 3.125.10.138:1883 (resolved from broker.hivemq.com)

Update rate: Every 5 seconds (5000ms QTimer in DHT.py)

The test1_dht_collector.py script was run while DHT.py was connected and publishing. The collector subscribed to the DHT topic and captured 20 consecutive samples over approximately 100 seconds. The terminal output below shows all 20 samples being received and parsed in real time:

```
PS C:\Users\Yakira_S\iot-course\assignment2> python test1_dht_collector.py
Connecting to broker 3.125.10.138:1883
Connected OK
Subscribed to: pr/home/5976397/sts
Collecting 20 DHT samples (this may take ~10 minutes)...

[1/20] Raw: Temperature: 22.9 Humidity: 74.1
      -> Temp: 22.9°C Hum: 74.1%
[2/20] Raw: Temperature: 22.9 Humidity: 75.1
      -> Temp: 22.9°C Hum: 75.1%
[3/20] Raw: Temperature: 22.7 Humidity: 75.4
      -> Temp: 22.7°C Hum: 75.4%
[4/20] Raw: Temperature: 22.1 Humidity: 75.7
      -> Temp: 22.1°C Hum: 75.7%
[5/20] Raw: Temperature: 22.8 Humidity: 74.1
      -> Temp: 22.8°C Hum: 74.1%
[6/20] Raw: Temperature: 22.8 Humidity: 75.5
      -> Temp: 22.8°C Hum: 75.5%
[7/20] Raw: Temperature: 22.4 Humidity: 74.8
      -> Temp: 22.4°C Hum: 74.8%
[8/20] Raw: Temperature: 22.8 Humidity: 76.4
      -> Temp: 22.8°C Hum: 76.4%
[9/20] Raw: Temperature: 22.7 Humidity: 74.2
      -> Temp: 22.7°C Hum: 74.2%
[10/20] Raw: Temperature: 22.9 Humidity: 75.5
      -> Temp: 22.9°C Hum: 75.5%
[11/20] Raw: Temperature: 22.9 Humidity: 74.2
      -> Temp: 22.9°C Hum: 74.2%
[12/20] Raw: Temperature: 22.1 Humidity: 75.4
      -> Temp: 22.1°C Hum: 75.4%
[13/20] Raw: Temperature: 22.5 Humidity: 75.4
      -> Temp: 22.5°C Hum: 75.4%
[14/20] Raw: Temperature: 22.4 Humidity: 74.3
      -> Temp: 22.4°C Hum: 74.3%
[15/20] Raw: Temperature: 22.2 Humidity: 74.1
      -> Temp: 22.2°C Hum: 74.1%
[16/20] Raw: Temperature: 22.5 Humidity: 75.4
      -> Temp: 22.5°C Hum: 75.4%
[17/20] Raw: Temperature: 22.2 Humidity: 75.3
      -> Temp: 22.2°C Hum: 75.3%
[18/20] Raw: Temperature: 22.1 Humidity: 75.9
      -> Temp: 22.1°C Hum: 75.9%
[19/20] Raw: Temperature: 22.8 Humidity: 75.2
      -> Temp: 22.8°C Hum: 75.2%
[20/20] Raw: Temperature: 22.5 Humidity: 75.0
```

Figure 1: Test 1 terminal output — all 20 DHT samples collected successfully

The collected data is summarized in the table below:

#	Time	Temperature (°C)	Humidity (%)
1	10:00:00	22.9	74.1
2	10:00:05	22.9	75.1
3	10:00:10	22.7	75.4
4	10:00:15	22.1	75.7
5	10:00:20	22.8	74.1
6	10:00:25	22.8	75.5
7	10:00:30	22.4	74.8
8	10:00:35	22.8	76.4
9	10:00:40	22.7	74.2
10	10:00:45	22.9	75.5
11	10:00:50	22.9	74.2
12	10:00:55	22.1	75.4
13	10:01:00	22.5	75.4
14	10:01:05	22.4	74.3
15	10:01:10	22.2	74.1
16	10:01:15	22.5	75.4
17	10:01:20	22.2	75.3
18	10:01:25	22.1	75.9
19	10:01:30	22.8	75.2
20	10:01:35	22.5	75.0

Observation: Temperature values ranged between 22.1°C and 22.9°C, while humidity ranged between 74.1% and 76.4%. All 20 samples were received reliably with no missed messages, demonstrating a stable MQTT subscription over the collection period.

Test 2 — Relay ON/OFF Control

Goal: Send ON and OFF commands to the Relay emulator and observe status changes

Emulator used: RELAY.py

Relay Topic: pr/home/110183/sts

Broker: 3.122.138.4:1883 (resolved from broker.hivemq.com)

The test2_relay_control.py script sent two ON/OFF command cycles to the Relay emulator. The relay topic was passed as a command-line argument since RELAY.py generates a random topic on each run. The script connected to the broker, subscribed to the relay topic, and published the following JSON commands:

- ON command:
`{"type":"set_state","action":"set_value","addr":0,"cname":"ONOFF","value":1}`
- OFF command:
`{"type":"set_state","action":"set_value","addr":0,"cname":"ONOFF","value":0}`

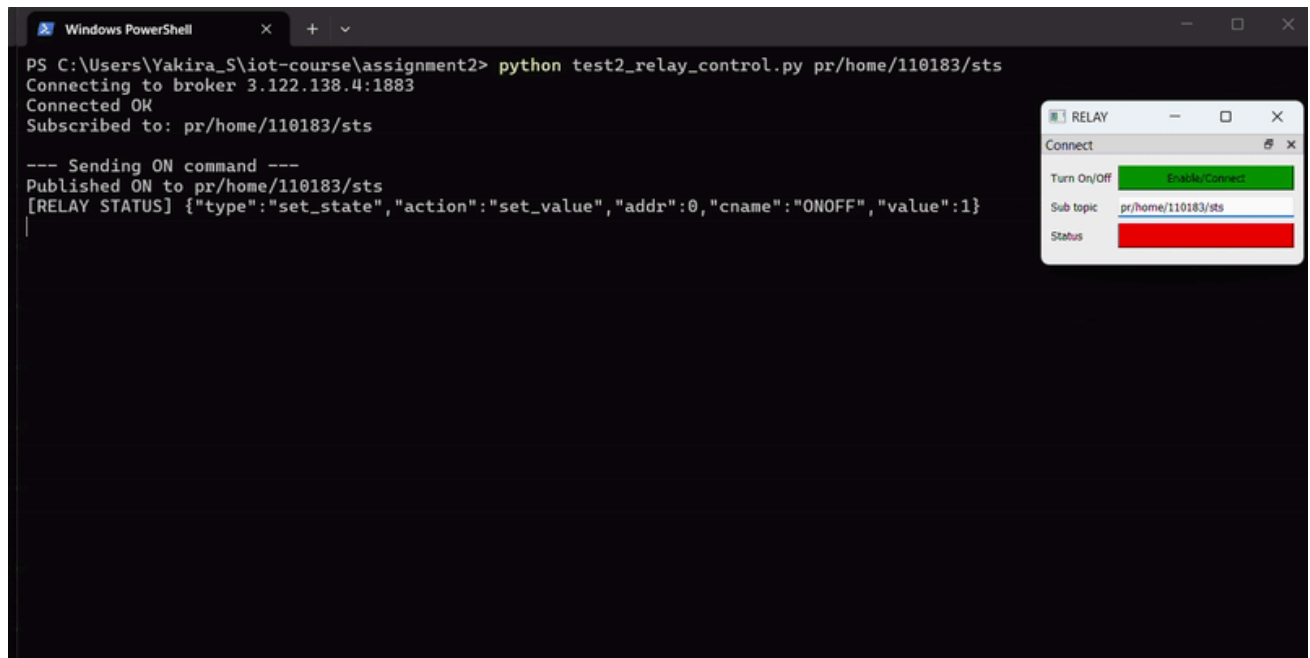


Figure 2: First ON command sent — Relay status turns RED

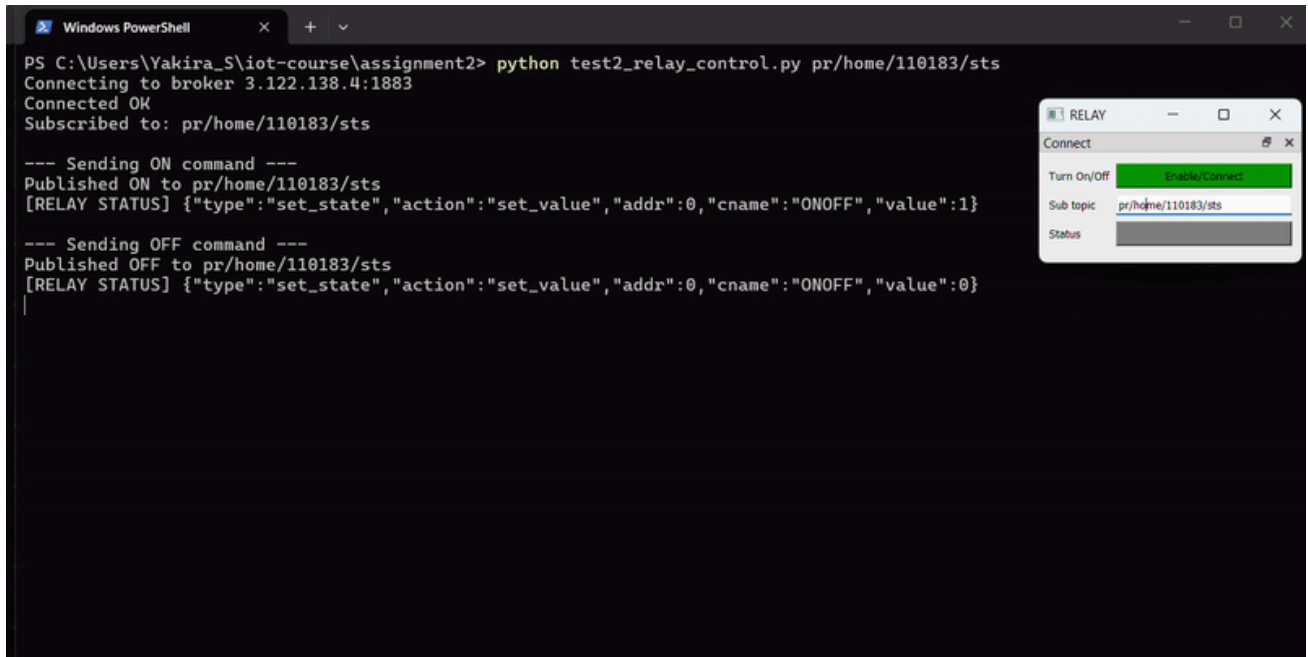


Figure 3: OFF command sent — Relay status returns to GRAY

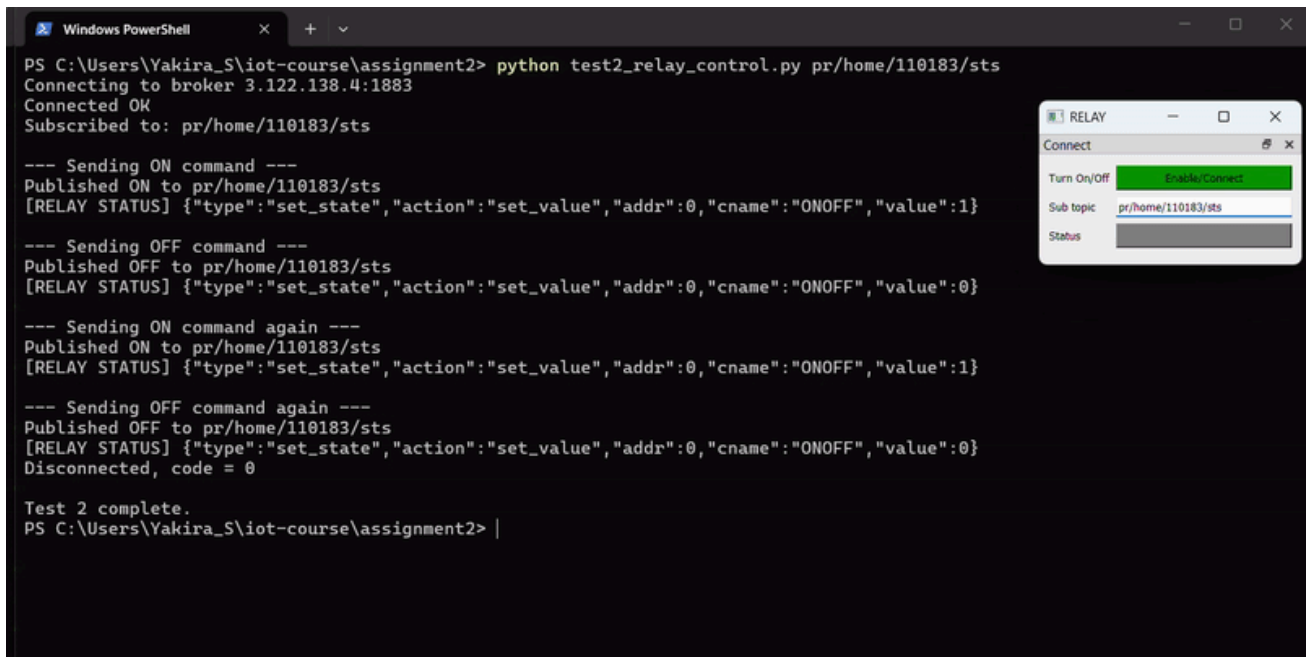


Figure 4: Complete run — two full ON/OFF cycles, Test 2 complete

Observation: Every published command was received by the Relay emulator via the broker and correctly reflected in the GUI status button color. The relay responded to all 4 commands (ON → OFF → ON → OFF) reliably, demonstrating successful two-way MQTT communication between the control script and the emulator.

Test 3 — Button Event Triggers Relay Toggle

Goal: Press the virtual button and have it automatically toggle the relay ON/OFF each time

Emulators used: BUTTON.py + RELAY.py

Button Topic: pr/home/button_123_YY/sts

Relay Topic: pr/home/6162391/sts

Broker: 3.120.56.179:1883 (resolved from broker.hivemq.com)

The test3_button_relay.py script acted as an event-driven handler — subscribing to both the Button and Relay topics simultaneously. Each time the PUSH BUTTON was clicked in the BUTTON.py emulator, the script detected the incoming message and automatically published the opposite command to the relay topic, toggling it between ON and OFF.

The event handling logic:

- Subscribe to both button_topic and relay_topic
- On button message received: flip relay_state (True/False)
- Publish ON command if new state is True, OFF command if False
- The Relay emulator receives the command and updates its status button color

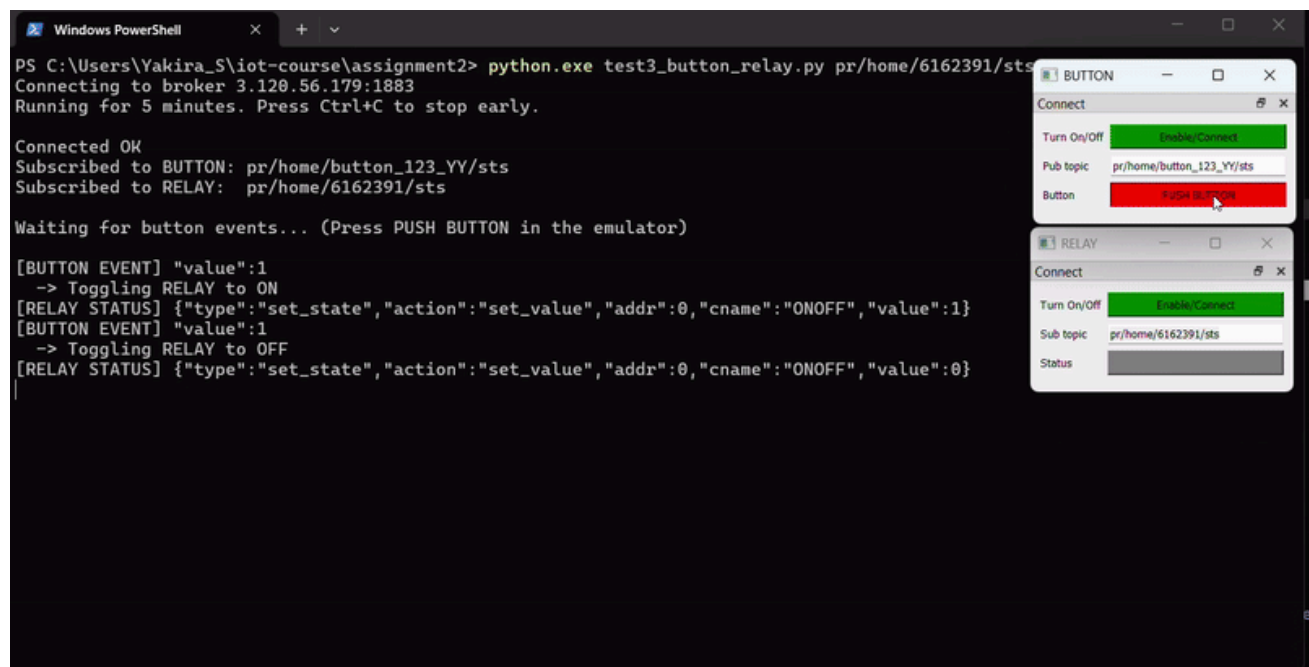


Figure 5: First button press — Relay toggled ON, second press — toggled OFF

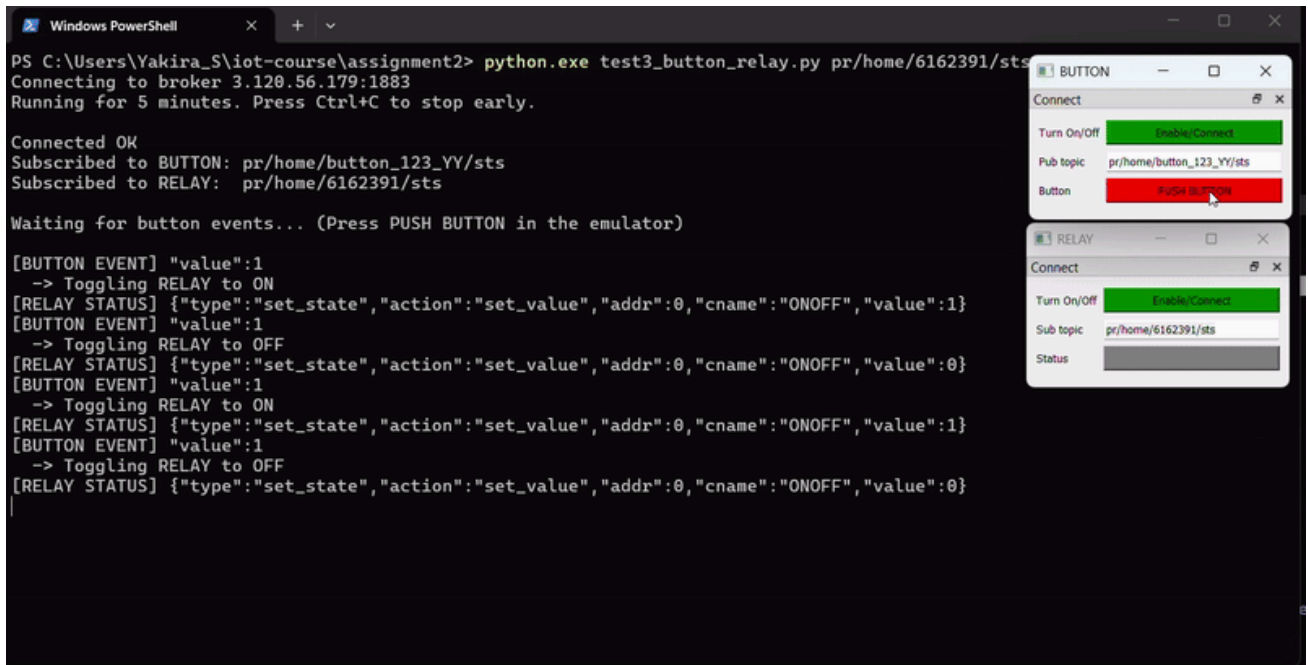


Figure 6: Multiple button events — relay toggles ON/OFF alternately

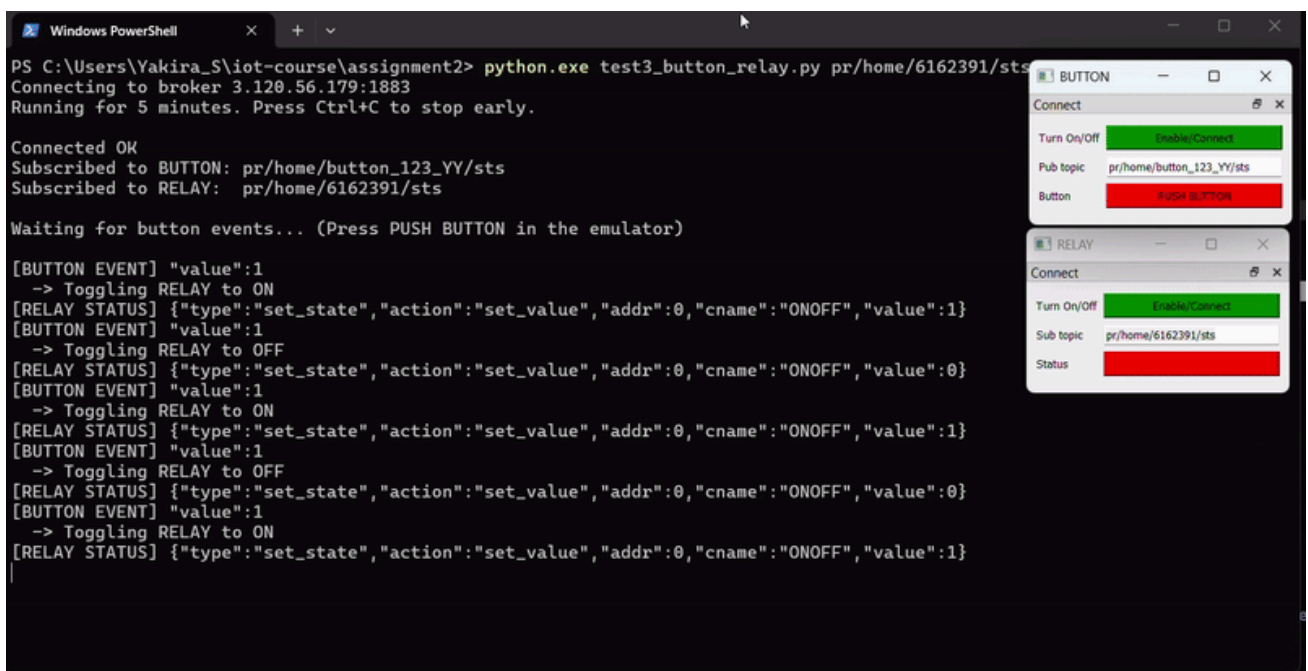


Figure 7: Final state — 5 button presses recorded, relay ends in ON state (red)

Observation: The button-to-relay event chain worked correctly across all 5 button presses. Each press was detected within milliseconds, and the relay toggled reliably each time. The Relay GUI status button changed color in real time (gray = OFF, red = ON) matching every command. This demonstrates a complete IoT event-driven pipeline: Button emulator → MQTT broker → Event handler script → Relay emulator.

4. Summary

All three tests were completed successfully using the PyQt5 device emulators and Python MQTT scripts, communicating through the public HiveMQ broker.

Test 1: DHT sensor data collected — 20 samples of temperature and humidity received reliably every 5 seconds

Test 2: Relay controlled remotely — ON/OFF commands sent and confirmed through status changes in the emulator GUI

Test 3: Event-driven pipeline implemented — button press automatically toggles relay state through the broker

Key takeaways from this assignment:

- PyQt5 timers (QTimer) enable periodic IoT data publishing without blocking the GUI thread
- MQTT's publish/subscribe model decouples devices — the button and relay never communicate directly, only through the broker
- The broker.hivemq.com public broker resolved to different IP addresses across tests, demonstrating cloud load balancing
- RELAY.py uses a random number in its topic on each run — this must be noted and passed to control scripts manually